API Contract: RAG-Diary

> The contract for every API endpoint: request method, response shape, error codes and authentication rules, so the development and test teams share one understanding.

> Thai version: API-CONTRACT.md

1. Document control

Item	Detail
Document	API Contract
Version	1.0
Prepared	15 October 2026
Author	Business Analyst / System Analyst (project owner)
Status	Reviewed, round 1 (25 October 2026)
Related documents	DATA-MODEL, SEQUENCE-DIAGRAMS, SRS

2. Purpose and scope

This document answers one question for each endpoint: what does it accept, what does it return, and how can it fail? It is the agreement between the frontend and backend teams.

Scope: 15 endpoints covering authentication, entry management and AI processing. Out of scope: deep data structures (in DATA-MODEL) and call sequences (in SEQUENCE-DIAGRAMS).

3. API conventions

Item	Value
Base URL	The browser calls its own origin at /api, and Next.js proxies to the Elysia server (target set by API_PROXY_TARGET)
Data format	JSON, except POST /chat which streams plain text
Authentication	Two httpOnly cookies (rag_access, rag_refresh) set by the server. An Authorization Bearer header is still accepted for scripts
Error shape	{"error": "description"}

Item	Value
Auth enforcement	Every endpoint in the notes and ai groups runs through authGuard

Environment variables: server side needs SUPABASE_URL, SUPABASE_ANON_KEY and GEMINI_API_KEY; the frontend needs API_PROXY_TARGET at build time to decide where /api/* is proxied. The code uses the anon key so RLS always applies; the service role key is a fallback only, because it bypasses RLS.

4. Endpoint summary

#	Method	Path	Auth	Description	Requirement
1	POST	/auth/register	No	Create an account	FR-01
2	POST	/auth/login	No	Log in	FR-02
3	POST	/auth/logout	No	Log out	FR-03
4	GET	/auth/me	Yes	Read the current user	FR-02, FR-03
5	GET	/notes	Yes	List the user's entries	FR-06
6	POST	/notes	Yes	Create an entry and index it	FR-04, FR-05
7	PATCH	/notes/:id	Yes	Edit an entry	FR-07
8	DELETE	/notes/:id	Yes	Delete one entry	FR-08
9	POST	/search/semantic	Yes	Semantic search	FR-09
10	POST	/chat	Yes	Chat with the entries (stream)	FR-10
11	POST	/ai/summarize	Yes	Mood summary and image prompt	FR-11
12	POST	/ai/imagine	Yes	Generate an image from the prompt	FR-12
13	POST	/auth/refresh	No (uses the refresh cookie)	Rotate the session	FR-02, FR-03
14	GET	/export	Yes	Export every entry as JSON	FR-13
15	DELETE	/data	Yes	Delete every entry of the user	FR-14

5. Endpoint details

5.1 POST /auth/register

Creates an account. No authentication required.

Request

```
{ "email": "user@example.com", "password": "secret123" }
```

Response 200

```
{
  "user": { "id": "uuid", "email": "user@example.com" },
  "needsEmailConfirmation": false
}
```

Sets the rag_access and rag_refresh cookies as httpOnly, SameSite=Lax and Secure in production. No token is returned in the body.

Errors: 400 when the email is invalid or the password is shorter than 6 characters.

5.2 POST /auth/login

Signs a user in. No authentication required.

Request

```
{ "email": "user@example.com", "password": "secret123" }
```

Response 200

```
{
  "user": { "id": "uuid", "email": "user@example.com" }
}
```

Sets both cookies as httpOnly. No token is returned in the body.

Errors: 401 when the credentials are wrong.

5.3 POST /auth/logout

Signs a user out. The server clears both cookies. Tokens are stateless, so nothing is revoked at the identity provider.

Response 200

```
{ "success": true }
```

5.4 GET /auth/me

Reads the current user. Authentication required.

Response 200

```
{ "user": { "id": "uuid", "email": "user@example.com" } }
```

Errors: 401 when there is no cookie or the token is invalid.

5.5 GET /notes

Lists all of the user's entries, newest first. Authentication required.

Response 200

```
[
  {
    "id": "uuid",
    "content": "Entry text",
    "tag": "General",
    "created_at": "2026-10-15T10:00:00Z"
  }
]
```

Returns only the columns the screen needs; the embedding and user_id are not sent (closes R-08).

5.6 POST /notes

Creates an entry and builds its semantic index automatically. Authentication required.

Request

```
{ "content": "Entry text", "tag": "Work" }
```

The tag is optional and defaults to General. Content is limited to 20,000 characters and the tag to 50. If the embedding fails, the text is still saved with an empty embedding, and that entry will not appear in search until it is saved again.

Response 200 returns the created row.

Errors: 401 when authentication fails.

5.7 PATCH /notes/:id

Edits an entry and rebuilds its semantic index. Authentication required.

Request

```
{ "content": "Edited text", "tag": "Work" }
```

Response 200

```
{ "success": true }
```

Errors: 401 when authentication fails.

5.8 DELETE /notes/:id

Deletes one entry by ID. Authentication required.

Response 200

```
{ "success": true }
```

Errors: 401 when authentication fails, and 404 when the entry does not exist or does not belong to the user.

5.9 POST /search/semantic

Semantic search. Authentication required.

Request

```
{ "query": "how I felt this week", "threshold": 0.6, "limit": 10 }
```

Threshold and limit are optional; defaults are 0.6 and 10. The query is 1 to 500 characters and limit is 1 to 50.

Response 200 returns the similar entries with their similarity score.

Errors: 401 when authentication fails. Provider failures are logged on the server and the client receives a generic 500.

5.10 POST /chat

Question and answer over the user's entries. Authentication required.

Request

```
{ "query": "how did I feel this week" }
```

Response 200 is a text stream, not JSON; the client must read it as a stream.

Errors: 401 when authentication fails. Provider failures are logged on the server and the client receives a generic 500.

5.11 POST /ai/summarize

Summarises the mood of the entries and proposes an image prompt. Authentication required.

Request

```
{ "notes": [ { "created_at": "2026-10-15T10:00:00Z", "content": "Entry text" } ]
}
```

The notes structure is validated: created_at optional, content 1 to 20,000 characters, at most 200 items. An empty list returns 400 (closes R-03).

Response 200

```
{ "result": "Summary text ...\nIMAGE_PROMPT: ..." }
```

5.12 POST /ai/imagine

Generates an image from the prompt. Authentication required. The prompt is 1 to 1,000 characters.

Request

```
{ "prompt": "a calm sunrise over a paper notebook" }
```

Response 200

```
{ "imageUrl": "https://..." }
```

5.13 POST /auth/refresh

Rotates the session by reading the refresh token from the rag_refresh cookie and setting a new pair. No new login required.

Request (no body needed from a browser)

```
{ "refresh_token": "..." }
```

The body is for non-browser clients; a browser sends the cookie automatically.

Response 200

```
{
  "user": { "id": "uuid", "email": "user@example.com" }
}
```

Sets both cookies again (rotation) and returns no token in the body.

Errors: 401 when the refresh token is invalid or expired, and both cookies are cleared so the client returns to the login page.

5.14 GET /export

Exports every entry of the signed-in user as JSON. Authentication required.

Response 200

```
{
  "exported_at": "2026-11-05T00:00:00.000Z",
  "count": 12,
  "entries": [
    { "id": "uuid", "content": "...", "tag": "General", "created_at": "..." }
  ]
}
```

5.15 DELETE /data

Deletes every entry of the signed-in user in one call. Authentication required.

Response 200

```
{ "success": true, "deleted": 12 }
```

Errors: 401 when not authenticated.

6. Error codes

Code	Meaning	Example situation
200	Success	The request completed
400	Invalid input	The email or password fails validation
401	Not authenticated	No cookie, or the token is invalid
403	Origin not allowed	The request Origin is not on the allow list (CSRF guard)
404	Not found	The entry does not exist or is not the user's
429	Too many requests	The per-user limit was exceeded (for example 20 chats per minute)
500	Internal error	An unexpected failure, including an AI provider error (proposal: split into 502 so the client can tell them apart)
503	AI service unavailable	The image provider returns 429 because the plan has no quota

Per-user limits (per minute): search 60, chat 20, summarize 20, imagine 5, notes 60. Counting is a fixed window and returns 429 when exceeded.

7. Observations and proposals

Topic	Detail	Proposal
POST /chat streams	The response is a text stream, not JSON	Specify the stream contract explicitly in the docs and tests
POST /ai/summarize had no type check	Fixed: the notes schema is validated and an empty list is rejected	Closed (R-03)
Rate limiting	Implemented in code (search 60, chat 20, summarize 20, imagine 5, notes 60 per minute)	Awaiting the migration before it is active (R-02)
GET /notes returned every column	Fixed: returns only id, content, tag, created_at	Closed (R-08)

8. References

Document	Relationship
DATA-MODEL	The structures these endpoints read and write

Document	Relationship
SEQUENCE-DIAGRAMS	The order in which these endpoints are called
SRS	The requirements these endpoints satisfy
TRACEABILITY-MATRIX	How endpoints map to requirements